Azure Service Bus

Azure Service Bus is a fully managed **enterprise message broker** provided by **Microsoft Azure**. It enables **reliable, asynchronous communication** between distributed applications and microservices using **queues** and **topics/subscriptions**.

At its core, Azure Service Bus decouples producers (senders) from consumers (receivers), allowing systems to scale independently, tolerate failures, and communicate without tight temporal or logical coupling.

Why Azure Service Bus Is Needed

In real-world distributed systems:

- Services should not directly depend on each other's availability.
- Traffic can be bursty and unpredictable.
- Long-running business workflows must be resilient to partial failures.
- Messages must not be lost, duplicated, or processed out of order.

Azure Service Bus addresses these concerns with **durable messaging**, **guaranteed delivery**, and **enterprise-grade reliability**.

Core Concepts

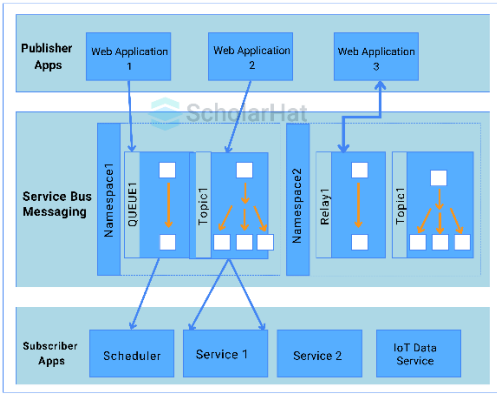
1. Namespace

A **namespace** is a logical container for messaging entities.

- Acts as a boundary for security, networking, and pricing
- Contains queues, topics, and subscriptions

---

2. Queues (Point-to-Point)



- **One sender → one receiver**
- Messages are stored until consumed
- Each message is processed by **only one consumer**

Key characteristics

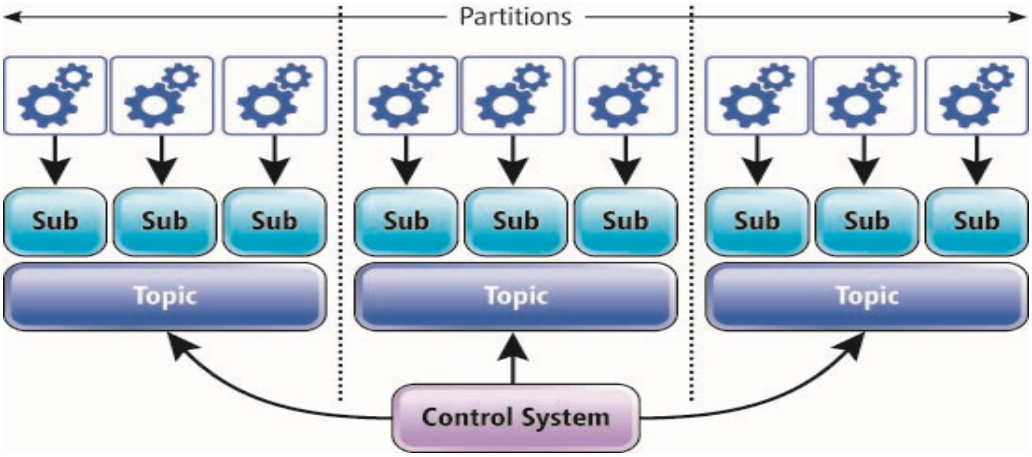
- FIFO (with sessions)
- Competing consumers supported
- Ideal for background processing and task distribution

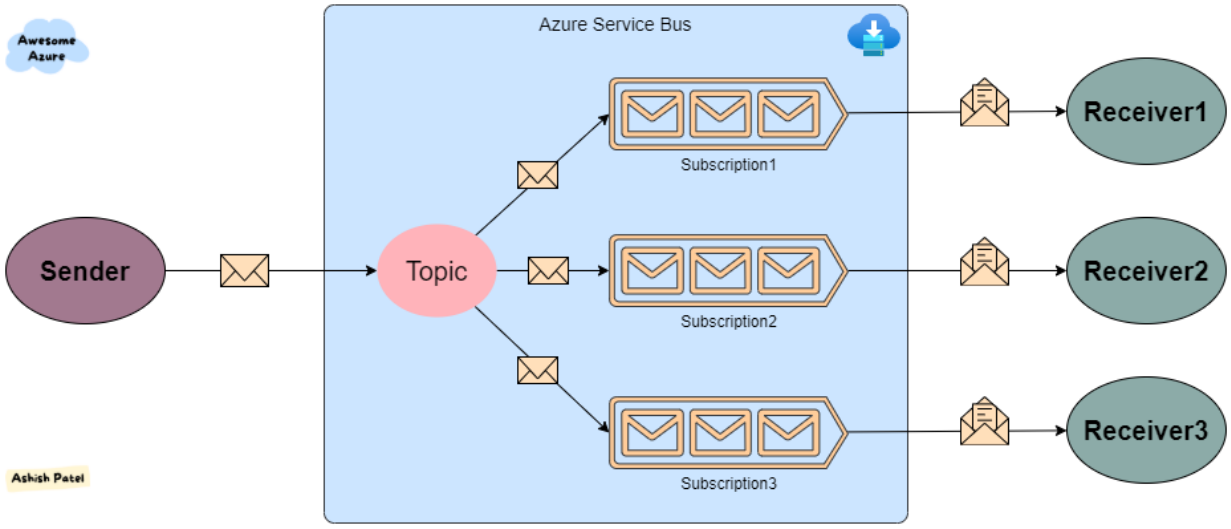
Example

Order API → Order Queue → Order Processor

---

3. Topics & Subscriptions (Publish/Subscribe)





- **One sender → many receivers**
- Sender publishes to a **topic**
- Each **subscription** receives its own copy

Advanced feature

- SQL-like filters on subscriptions

Example

OrderCreated Topic

├── Billing Subscription

├── Inventory Subscription

└── Notification Subscription

---

4. Messages

A message consists of:

- **Body** (JSON, XML, binary)
- **System properties** (MessageId, CorrelationId)
- **Custom properties** (key–value metadata)

Messages are **durable** and persisted until acknowledged.

Delivery Guarantees

Feature	Description
At-least-once delivery	Message is never lost
Peek-Lock	Message locked during processing
Complete	Confirms successful processing
Abandon	Returns message to queue

Common Use Cases (Real-World)

1. Microservices Communication

Problem: Tight coupling via HTTP
Solution: Event-driven communication

Order Service → Topic → Inventory / Payment / Email

Benefits:

- Loose coupling
- Independent scaling
- Failure isolation

---

2. Order Processing System

- API receives order
- Publishes message to queue
- Background worker processes order asynchronously

Improves:

- API responsiveness
- Throughput

- Reliability

---

3. Saga Pattern (Distributed Transactions)

Azure Service Bus is widely used for **choreography-based sagas**:

- OrderCreated → PaymentProcessed → InventoryReserved
- Each step publishes an event
- Compensating events on failure

---

4. Integration Between Systems

- Legacy app → Service Bus → Cloud microservices
- Secure, reliable integration layer

Azure Service Bus vs Other Azure Messaging Services

Service	**Best For**
Azure Service Bus	Enterprise workflows, commands, transactions
Event Grid	Reactive events, UI triggers
Event Hubs	Big data streaming, telemetry

Rule of thumb:

- **Commands & workflows → Service Bus**
- **Events & notifications → Event Grid**
- **High-throughput streams → Event Hubs**

When You Should Use Azure Service Bus

Use it when you need:

- Guaranteed message delivery

- Decoupled microservices
- Ordered processing
- Transactional messaging
- Reliable enterprise workflows

Summary

Azure Service Bus is a **foundational building block** for cloud-native and microservices architectures on Azure. It enables reliable, scalable, and maintainable systems by introducing **asynchronous, message-driven communication**.